

Assignment # 1

Question 1

- Merge Sort = $32n \lg n$
- Insertion Sort = $4n^2$

The values of n which can beat merge sort should satisfy the following inequality:

$$4n^2 < 32 n \log n$$

$$n^2 < 8 n \log n$$

$$n < 8 \log n$$

$$\frac{n}{\log n} < 8$$

That is all values of $n < 43.5$

Question 2

- Algo 1 = $100n^2$
- Algo 2 = 2^n

The smallest value of n that satisfies the following equation would be our answer:

$$100n^2 < 2^n$$

$$\log 100n^2 < n$$

That is the smallest value of **14.4**

Question 3

(a)

```
1 int s, i, n;          ---- 1
2 cin >> n;             ---- 1
3 s = 0;                ---- 1
4 for(i=n; i>=1; i--)   ---- n + 1
5     s++;              ---- n * 1
```

$$\text{Total} = 1 + 1 + 1 + (n + 1) + n$$

$$T(n) = 2n + 4$$

$$O(n)$$

- **$T(n) \leq c f(n)$**

- $2n + 4 \leq cn$
- $2 + \frac{4}{n} \leq c$
 - Let $n = 1$
 - $2 + 4 \leq c$
 - $6 \leq c$, therefore, c needs to be greater than or equal to 6 for every $n \geq 1$
- $2n + 4 \leq 6n$
 - That is for $n \geq n_0$ where $n_0 = 1$
 - This condition satisfies for all values of $n \geq 1$
- Hence, we have proved that $T(n)$ is $O(n)$

(b)

```
1 int sum, i, j, n;      ----- 1
2 sum = 0;               ----- 1
3 cin >> n;              ----- 1
4 for(i=1; i<n; i=i*2)    ----- log2(n)
5     for(j=1; j<n; j=j*2) ----- log2(n)-1 * log2(n)
6         sum++;          ----- log2(n)-1 * log2(n)-1 * 1
```

$$\begin{aligned}
\text{Total} &= 1 + 1 + 1 + \log(n) + \log(n) * (\log(n) - 1) + (\log(n) - 1) * (\log(n) - 1) * 1 \\
&= 3 + \log(n) + \log(n)^2 - \log(n) + (\log(n) - 1)^2 \\
&= 3 + \log(n)^2 + \log(n)^2 + 1 - 2\log(n) \\
T(n) &= 4 + 2*\log(n)^2 - 2\log(n) \\
&O(\log(n)^2)
\end{aligned}$$

- **$T(n) \leq c f(n)$**

- $4 + 2\log^2(n) - 2\log(n) \leq c \log^2(n)$

- $\frac{4}{\log^2 n} + 2 - \frac{2}{\log n} \leq c$

- Let $n = 2$

- $\frac{4}{\log^2 2} + 2 - \frac{2}{\log 2} \leq c$

- $\frac{4}{1} + 2 - \frac{2}{1} \leq c$

- Therefore for $n \geq 2$, c should be ≥ 4

-

$$4 + 2\log^2 n - 2\log n \leq 4\log^2 n$$

- For every $n \geq n_0$ where $n_0 = 2$

- Therefore, $4+2\log^2(n)$ is $O(\log^2(n))$

(c)

```

1 int x = 0, j=n;      ----- 1
2 while(j>0) {         ----- log4(n) + 1
3     x += j*3;         ----- log4(n) * 1
4     j /= 4;           ----- log4(n) * 1
5 }
```

$$\text{Total} = 1 + \log_4(n) + 1 + \log_4(n) + \log_4(n)$$

$$T(n) = 2 + 3*\log_4(n)$$

$$O(\log_4(n))$$

- **$T(n) \leq c f(n)$**

- $2 + 3\log_4(n) \leq c \log_4(n)$

- $\frac{2}{\log_4 n} + 3 \leq c$

- Let $n = 4$

- $\frac{2}{\log_4 4} + 3 \leq c$
- $\frac{2}{1} + 3 \leq c$
- Therefore c should be at least ≥ 5 for every $n \geq 4$
 - $\frac{2}{\log_4 n} + 3 \leq 5$
 - For every $n \geq n_0$ where $n_0 = 4$
- Hence, $2 + 3\log_4(n)$ is Big Oh of $\log_4 n$

Question 4

PseudoCode

```

1 for(int i=0; i<n-1; i++){
2     int x = i
3     for(int j=i+1; j<n; j++){
4         if (A[x] > A[j]){
5             x = j
6         }
7     }
8     if (i != x){
9         int temp = A[i]
10        A[i] = A[x]
11        A[x] = temp
12    }
13 }

```

Why is it running for only n-1 times?

The outer loop iterator is being used to set the values of indexes respectively by swapping with the smaller elements. The inner loop needs to run n times to compare with each value but as for the outer loop, since the time, it had reached n th value, that n th value must already be in correct position by the swapping done in earlier iterations. Hence, there should not be a need to iterate over the n th time.

Best Case	Worst Case
$\Theta(n^2)$	$\Theta(n^2)$

Question 5

Prove that $T(n)$ is $\Theta(n^3)$

$$T(n) = \frac{1}{8}n^3 - 5n^2$$

We need to prove that:

$$c_1(n^3) \leq \frac{1}{8}n^3 - 5n^2 \leq c_2(n^3)$$

$$c_1 \leq \frac{1}{8} - \frac{5}{n} \leq c_2$$

Let $n = 5$

$$c_1 \leq \frac{1}{8} - \frac{5}{5} \leq c_2$$

$$c_1 \leq -\frac{7}{8} \leq c_2$$

$\therefore c_1$ should have a value equal or less than $-\frac{7}{8}$ for $n \geq 5$

Also, c_2 should have a value greater than $-\frac{7}{8}$ for $n \geq 5$.

Since $\frac{1}{8}n^3$ will eventually get large and the value will get positive thus we will keep a positive value as c_2 for our upper bound

Thus let $c_1 = -\frac{7}{8}$ and $c_2 = \frac{1}{8}$

$$-\frac{7}{8}(n^3) \leq \frac{1}{8}n^3 - 5n^2 \leq \frac{1}{8}(n^3)$$

For every $n \geq n_0$ where $n_0 = 5$

Hence $T(n)$ is $\Theta(n^3)$

Question 6

```
Function Mystery (n)
{
```

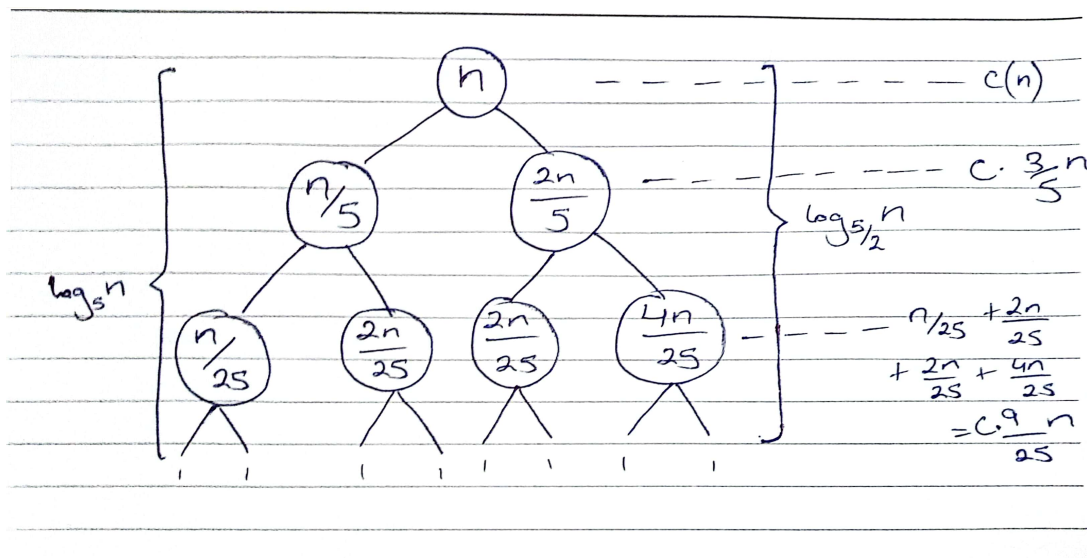
```

5   If (n > 1)
6   {
7       Print "hello"
8       Mystery(n/5)
9       For (i=1 .... n)
10          Print "world"
11      Mystery(2n/5)
12  }

```

The equation for this recursive function would be:

$$T(n) = T\left(\frac{n}{5}\right) + T\left(\frac{2n}{5}\right) + O(n)$$



Recursion Tree of the above Function

We can observe that at each level there are $c \left(\frac{3}{5}\right)^k n$ count of instructions being executed where k is the level count

The following series can be observed:

$$n\left(\left(\frac{3}{5}\right)^0 + \left(\frac{3}{5}\right)^1 + \left(\frac{3}{5}\right)^2 + \left(\frac{3}{5}\right)^3 + \dots + \left(\frac{3}{5}\right)^{\log_{\frac{5}{2}} n}\right)$$

Where $\log_{\frac{5}{2}} n$ is the height of the right most branch of the tree (the longest branch, as being divided by $\frac{5}{2}$ each time)

$$\therefore \sum_{k=0}^{\log_{\frac{5}{2}} n} \left(\frac{3}{5}\right)^k$$

To Solve This:

$$\sum_{k=0}^{\log_{\frac{5}{2}} n} \left(\frac{3}{5}\right)^k < \sum_{k=0}^{\infty} \left(\frac{3}{5}\right)^k$$

By applying summation formula:

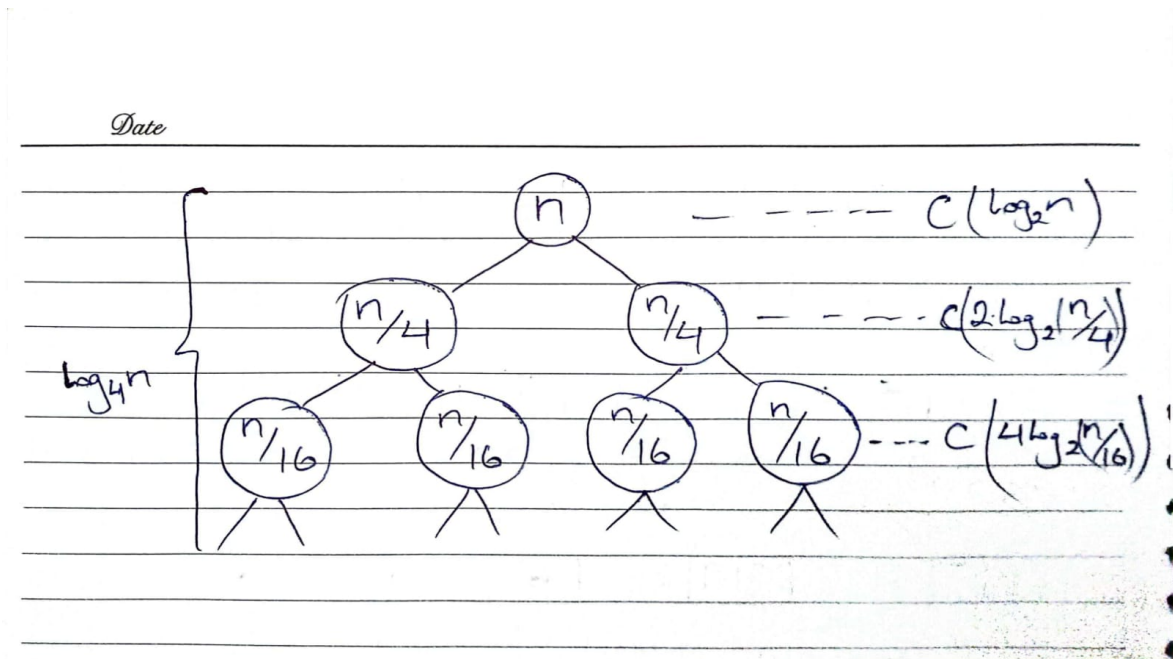
$$n \left(\frac{1}{1 - \frac{3}{5}} \right) = 2.5n$$

Hence, we have $O(n)$ as our runtime for this algorithm

Question 7

(a)

$$T(n) = 2T\left(\frac{n}{4}\right) + O(\lg n)$$



Solution

Date

$$= \lg n + 2 \lg(n/4) + 4 \lg(n/16) + \dots$$

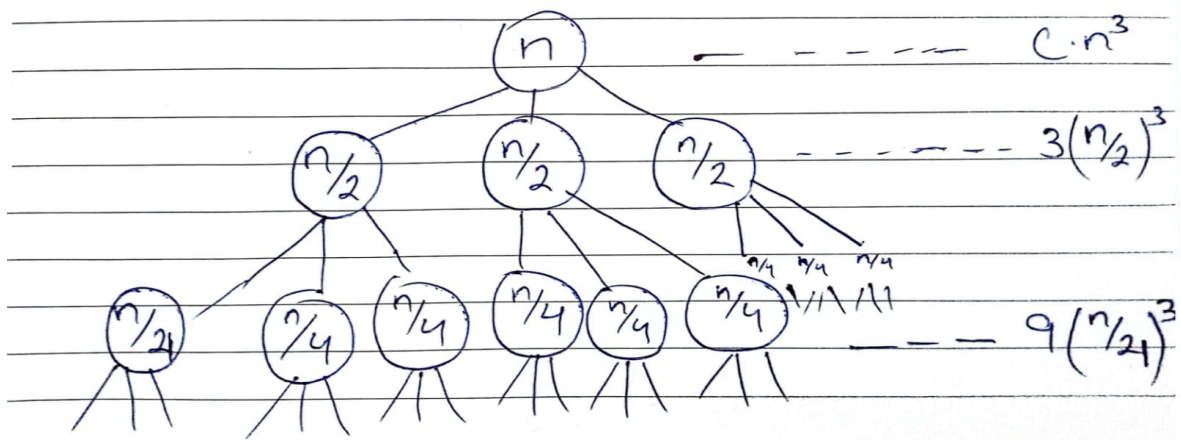
$$= 2^0 \lg(n/4^0) + 2^1 \lg(n/4^1) + 2^2 \lg(n/4^2) + \dots + 2^{\lg_4 n} \lg(n/4^{\lg_4 n})$$

$$= \sum_{i=0}^{\lg_4 n} 2^i \lg(n/4^i)$$

(b)

$$T(n) = 3T\left(\frac{n}{2}\right) + O(n^3)$$

Date _____

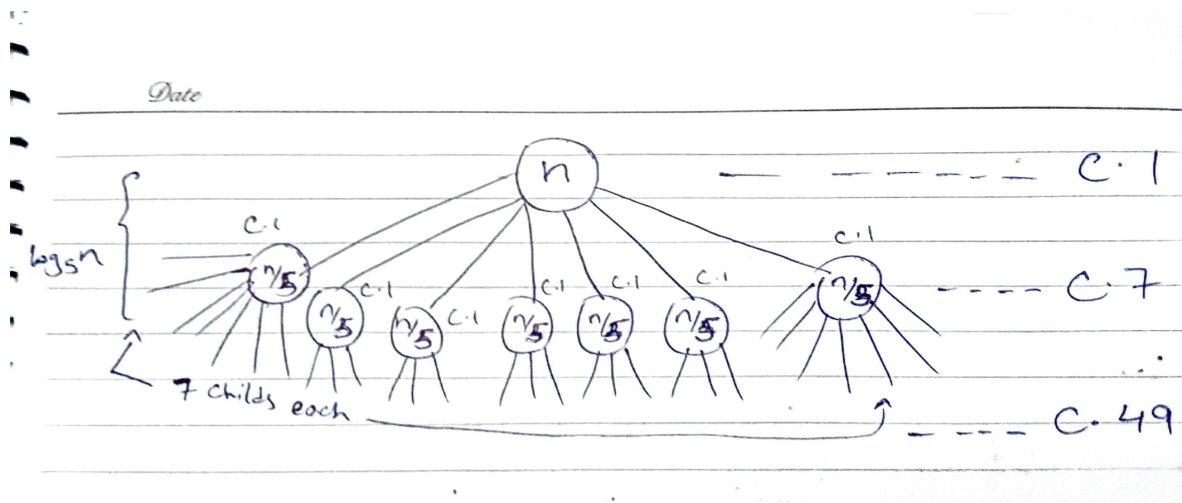


Solution

$$\begin{aligned}
 &= 3^0 \left(\frac{n}{2^0}\right)^3 + 3^1 \left(\frac{n}{2^1}\right)^3 + 3^2 \left(\frac{n}{2^2}\right)^3 \\
 &\quad + \dots + 3^{\log_2 n} \left(\frac{n}{2^{\log_2 n}}\right)^3 \\
 &= n^3 \left[\sum_{i=0}^{\log_2 n} \left(\frac{3^i}{2^{3i}}\right) \right] = n^3 \left[\sum_{i=0}^{\log_2 n} \left(\frac{3}{2^3}\right)^i \right] \\
 &= n^3 \left(\frac{1}{1 - 3/8} \right) = 1.6n^3 \\
 &\quad = O(n^3)
 \end{aligned}$$

(c)

$$T(n) = 7T\left(\frac{n}{5}\right) + \theta(1)$$

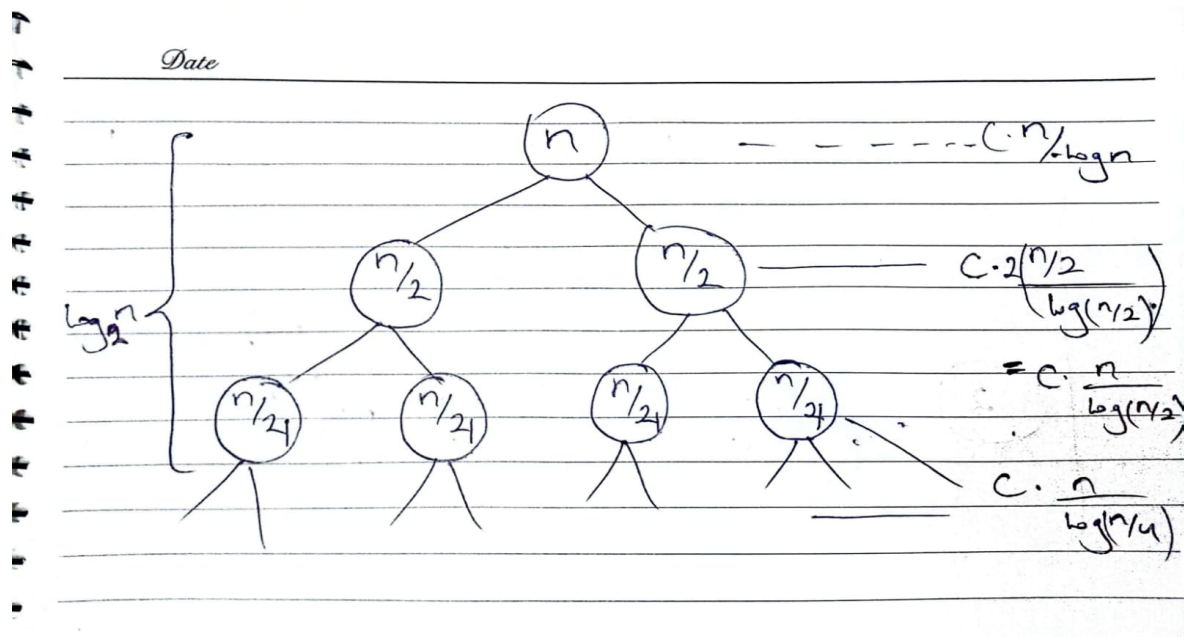


Solution

$$\begin{aligned}
 &= 1 + 7 + 49 + \dots \\
 &= 7^0 + 7^1 + 7^2 + \dots + 7^{\log_5 n} \\
 &= \sum_{i=0}^{\log_5 n} 7^i \\
 &= \frac{7^{\log_5 n + 1} - 1}{7 - 1} = \frac{7^{\log_5 n} \cdot 7 - 1}{6} \\
 &= \frac{n^{\log_5 7} \cdot 7 - 1}{6} = \frac{n^{1.2} \times 7 - 1}{6} \\
 &= O(n^{1.2})
 \end{aligned}$$

(d)

$$T(n) = 2T\left(\frac{n}{2}\right) + O\left(\frac{n}{\log n}\right)$$



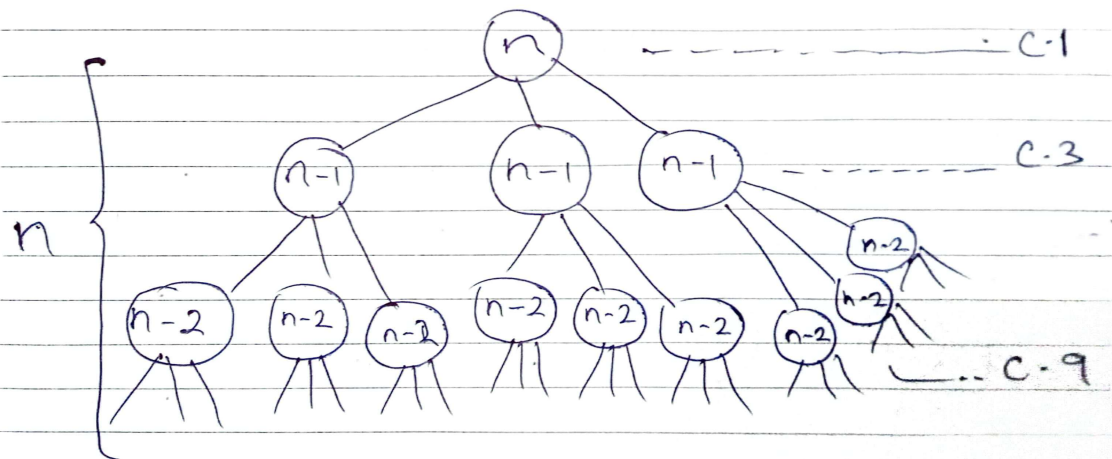
Solution

$$\begin{aligned}
 &= \frac{n}{\log n} + \frac{n}{\log(n/2)} + \frac{n}{\log(n/4)} + \dots \\
 &= n \left[\sum_{i=0}^{\log_2 n - 1} \frac{1}{\log(n/2^i)} \right] \\
 &= n \left[\sum \frac{1}{\log n - \log 2^i} \right] = n \left[\sum \frac{1}{\log n - i} \right] \\
 &\text{Harmonic Series} \\
 &= n \left[\frac{1}{\log n - 0} + \frac{1}{\log n - 1} + \frac{1}{\log n - 2} + \dots + \frac{1}{\log n - (\log n - 1)} \right] \\
 &= n \left[\log(\log n) \right] = n \log(\log n)
 \end{aligned}$$

(e)

$$T(n) = 3T(n-1) + \theta(1)$$

Date



Solution

$$= 1 + 3 + 9 + \dots$$

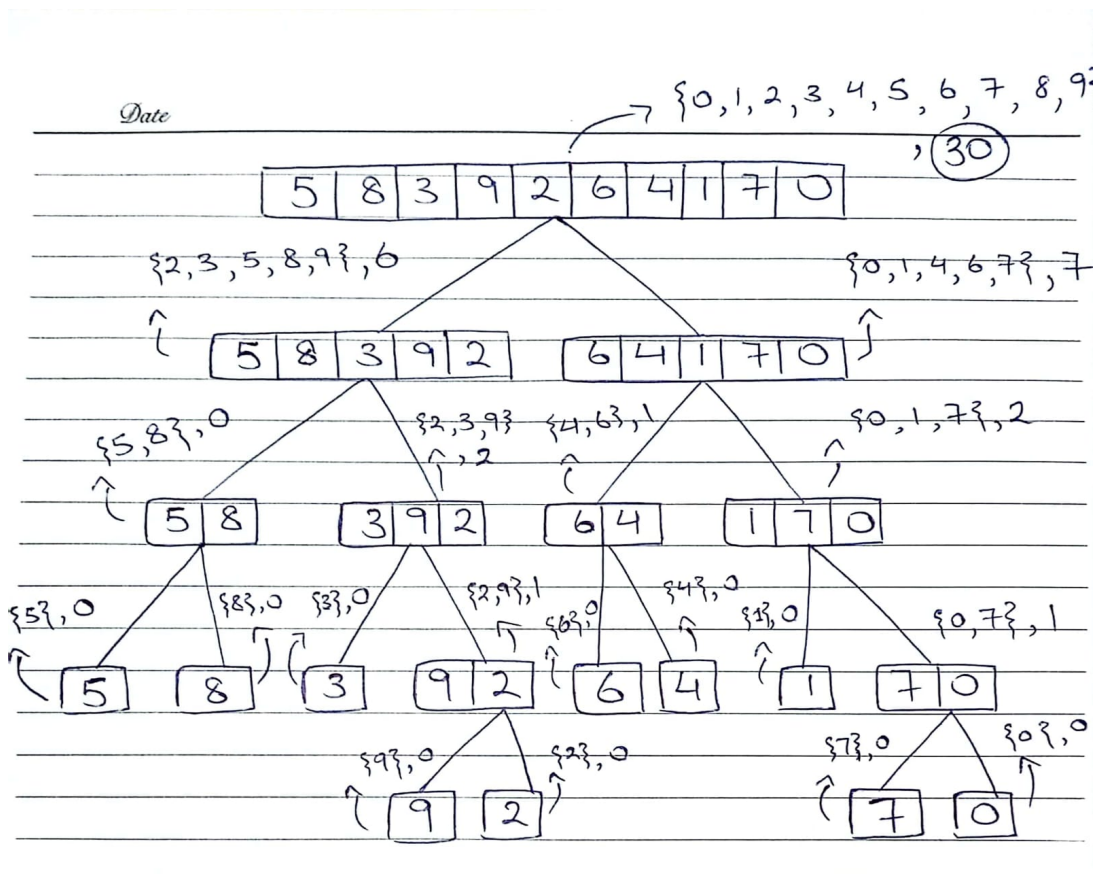
$$= 3^0 + 3^1 + 3^2 + \dots + 3^n$$

$$= \frac{3^{n+1} - 1}{3 - 1} = \frac{3^{n+1} - 1}{2}$$

$$= O(3^n)$$

Question 8

Inversion Pairs Algorithm



Count Inversion Pairs Algorithm - Dry Run

{ } Curly Braces represent the return value, which is an array in our case, of each recursive call
The value followed by it is the count of the split inversion pairs

The given array has a **total count of inversion pairs of 30**